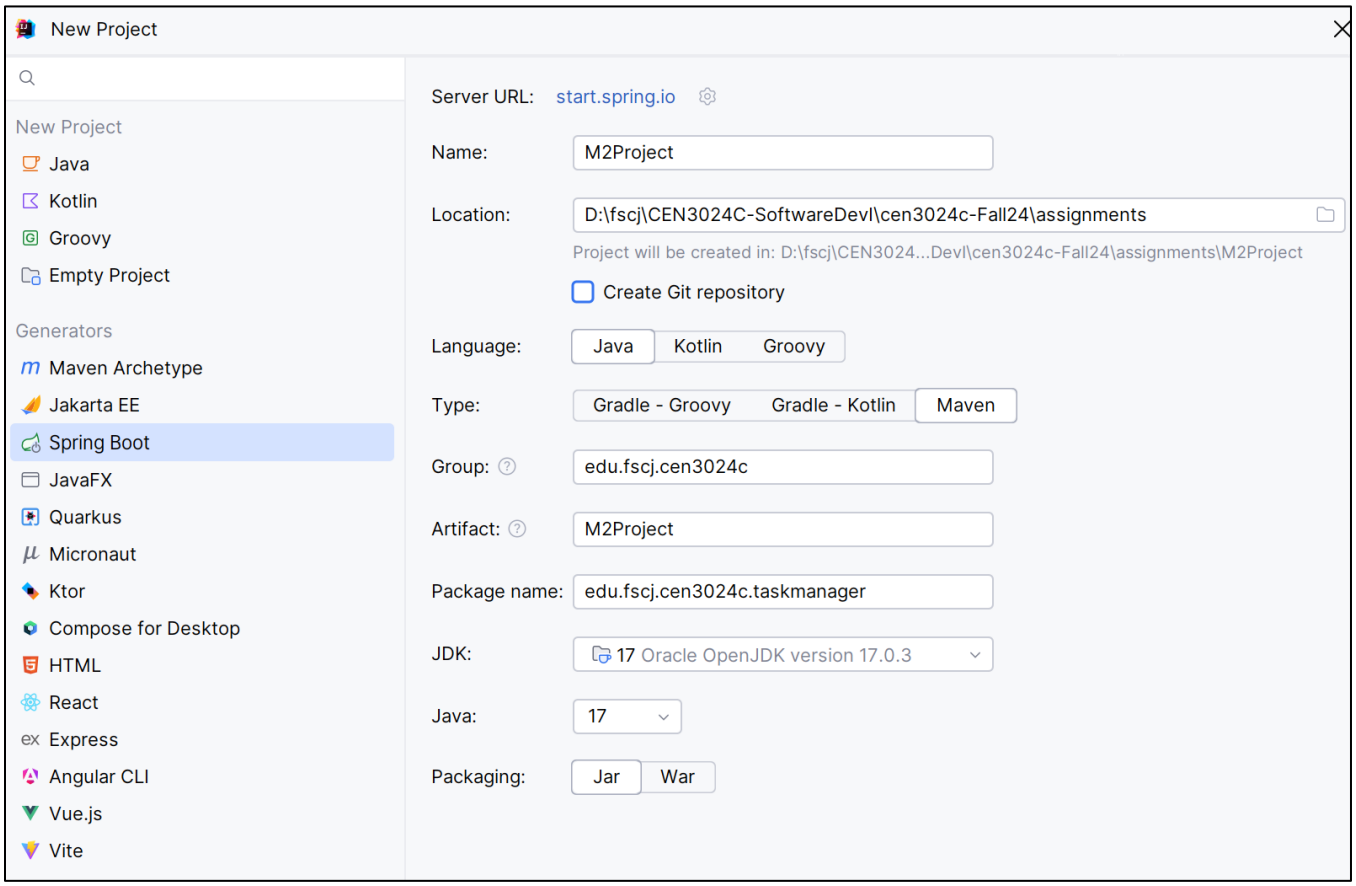
CEN3024C Module 2 Project

For this project we will practice using PostgreSQL and IntelliJ w/Spring Data JPA to create a simple task management application which includes a database, repository class, service class, and controller class with API endpoints. The application will use Spring Web which provides a basic MVC configuration.

- Start by creating your project using the IntelliJ Spring Boot generator. Select Java and Maven. Your group should be the "upper level" of your package (e.g., edu.fscj.cen3024c) and the package should extend that to indicate the project.
- Select Java 17 and Jar packaging.



Spring Boot Dependencies:

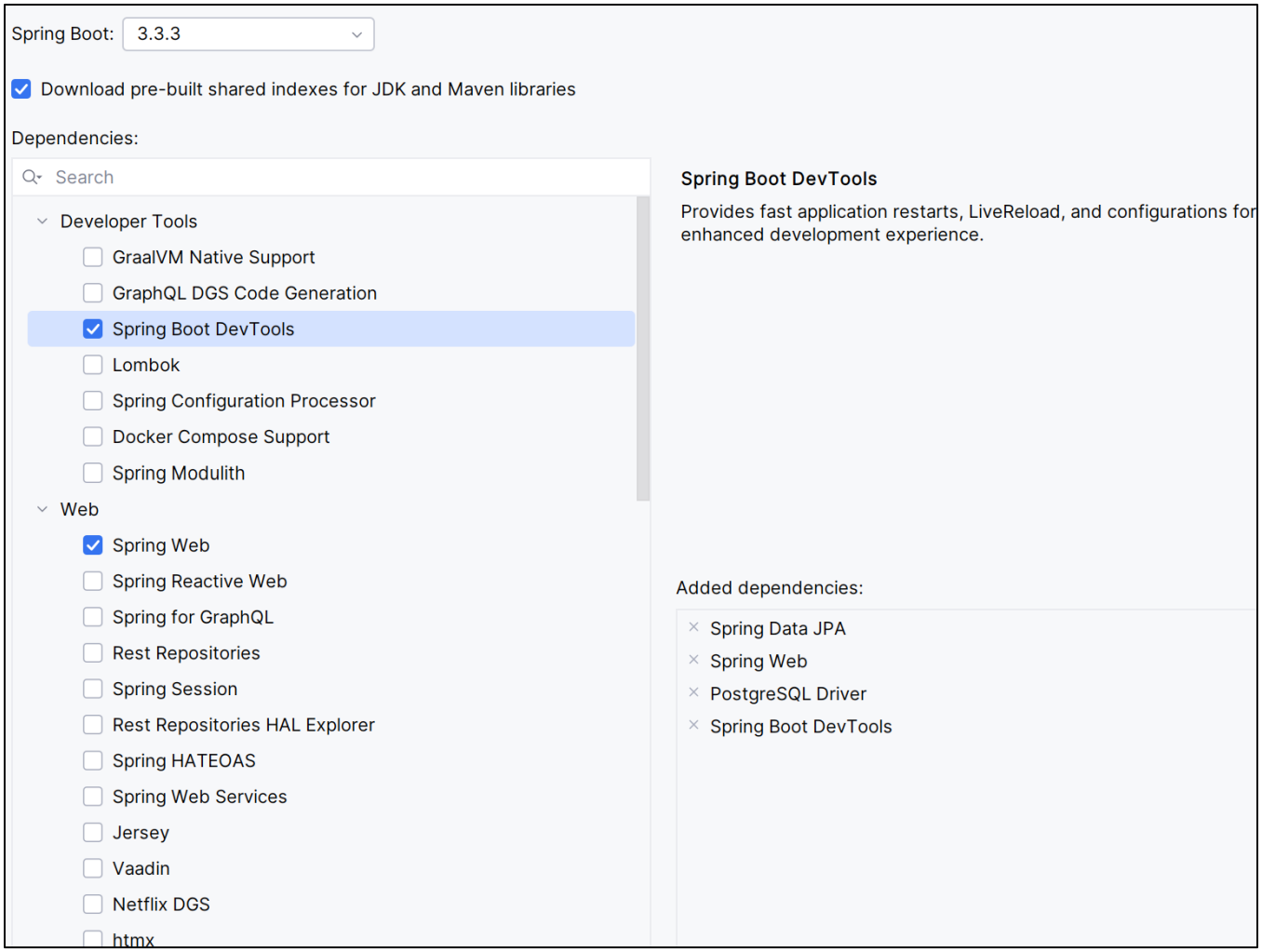
Developer Tools/Spring Boot DevTools

(provides development-time features like automatic restarts, live reload, and faster application restarts)

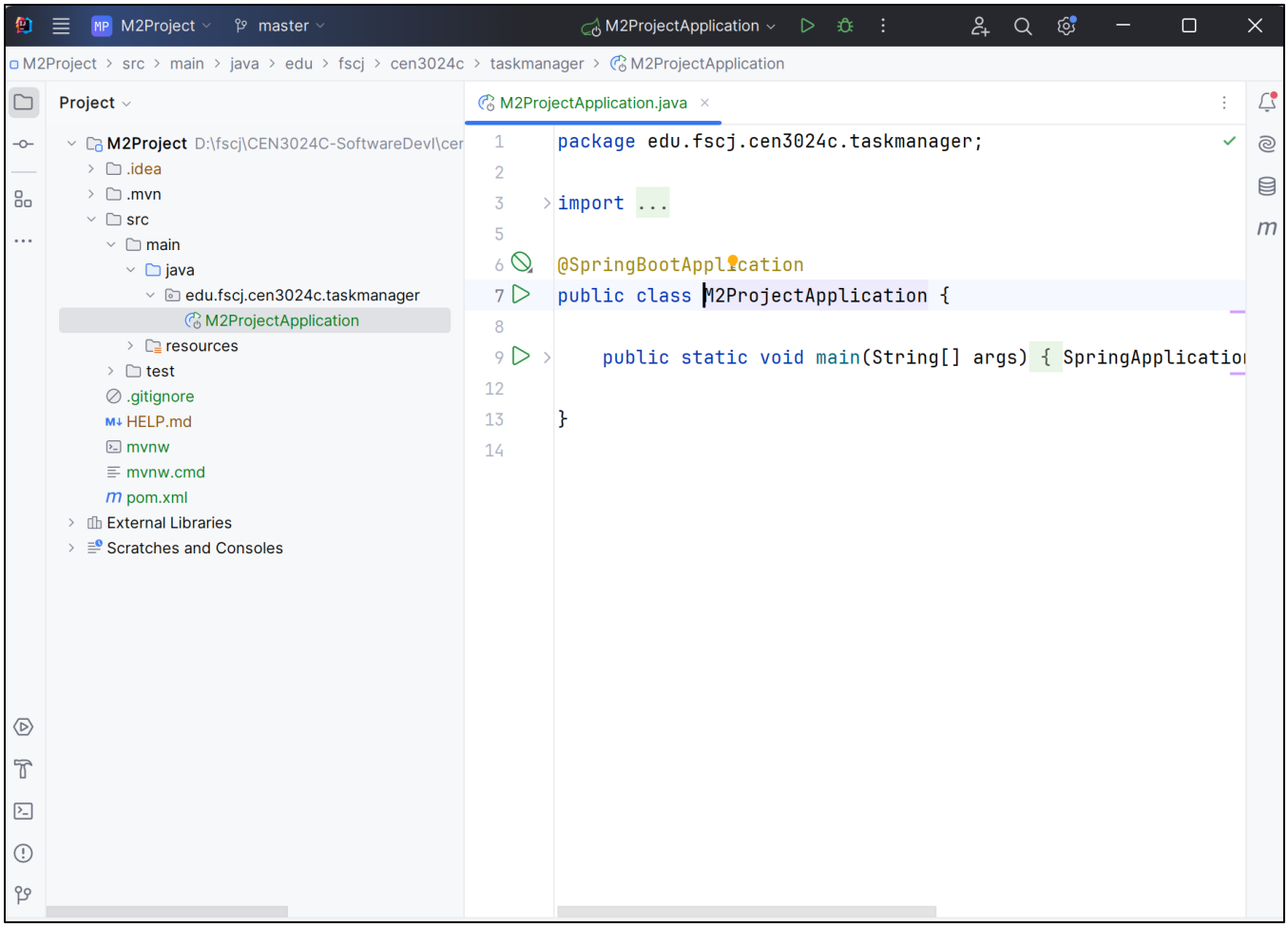
Web/Spring Web

SQL/Spring Data JPA

SQL/PostgreSQL Driver



IntelliJ creates the application class for you:

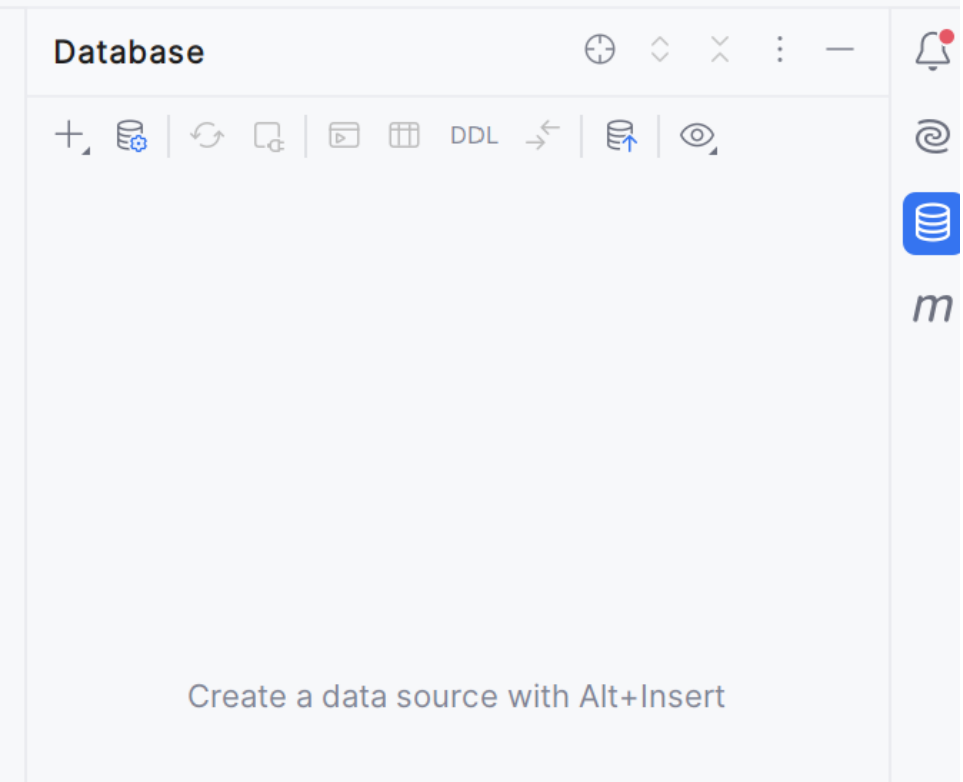


Here are some PostgreSQL tips from a previous semester:
Names which use upper case must be in double quotes
Use lower case (default) for table names so JPA/Hibernate annotation works
c:\"Program Files"\PostgreSQL\16\bin\psql -U postgres -d dbname
(postgres is default DB)
or just
c:\"Program Files"\PostgreSQL\16\bin\psql -U postgres
(to create/drop a DB)
P@ssw0rd1 (on Horizon)
\l to list databases
\c dbname to connect to a db
\dt to list tables and relationships (reports everything as "relations")
\dt+ for extended details
\dn to list schemas
\q to quit

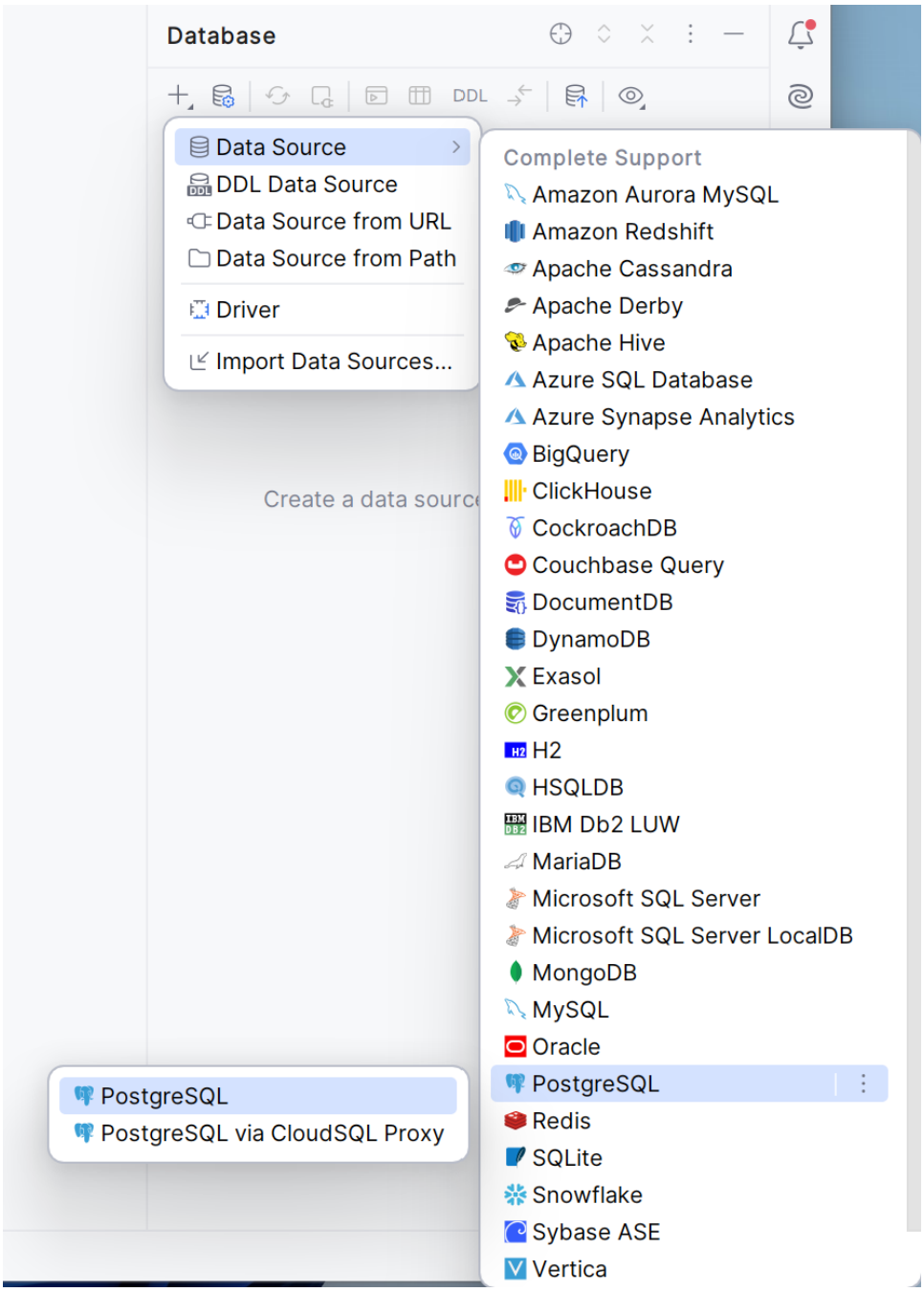
- Create the database using psql:

```
D:\> c:\"Program Files"\PostgreSQL\16\bin\psql -U postgres -d postgres
Password for user postgres
psql (16.1)
WARNING: Console code page (437) differs from Windows code page (1252)
         8-bit characters might not work correctly. See psql reference
         page "Notes for Windows users" for details.
Type "help" for help.

postgres=# \l
   List of databases
   Name    |  Owner   ...
 postgres  | postgres ...
...
(3 rows)

postgres=# CREATE DATABASE taskmanager_db;
CREATE DATABASE

postgres=# \l
      List of databases
      Name       |  Owner   ...
 postgres        | postgres ...
 taskmanager_db | postgres ...
 ...
(4 rows)

postgres=# \q

D:\fscj\CEN3024C-SoftwareDevI\cen3024c-Fall24\assignments>
```

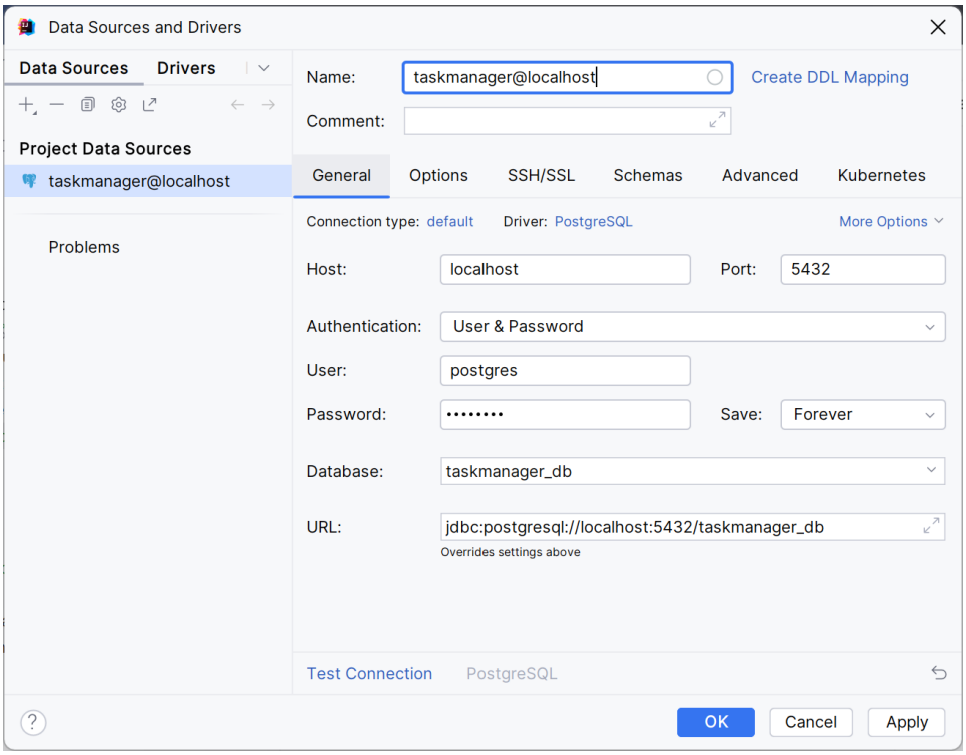
- Open the Database tool in IntelliJ:



- Create a new data source using PostgreSQL:



(**note:** the Name: field refers to the name of the connection; specify the actual database name in the Database field)

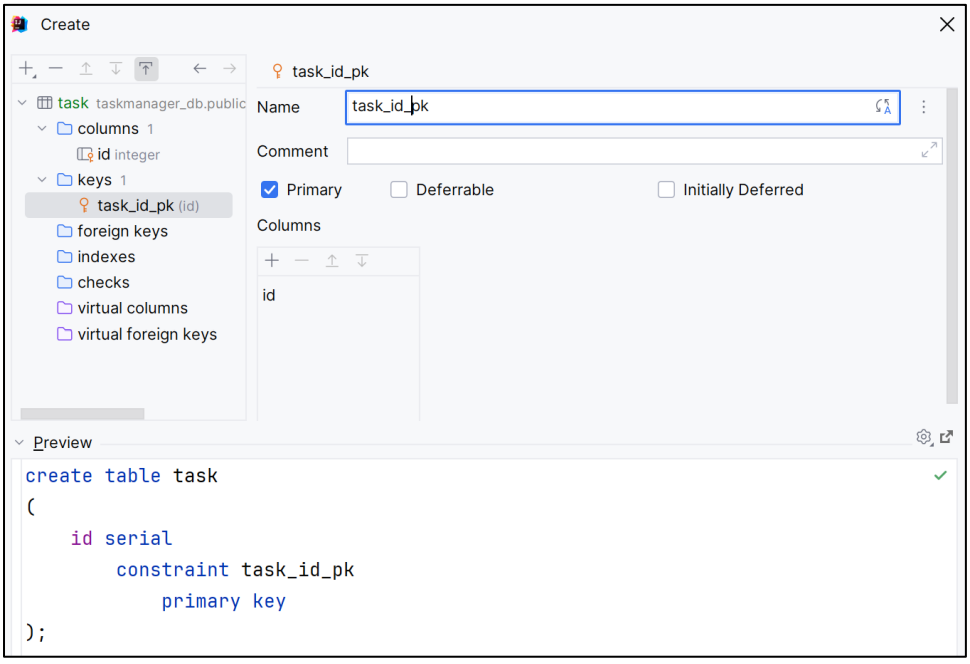


- Create the following table in the new taskmanager_db database:

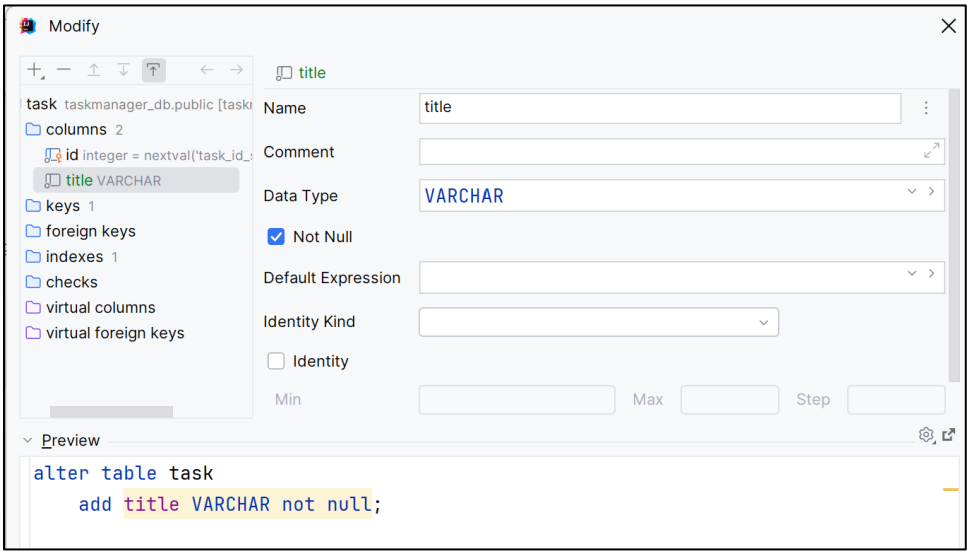
Table Name: task

Columns:

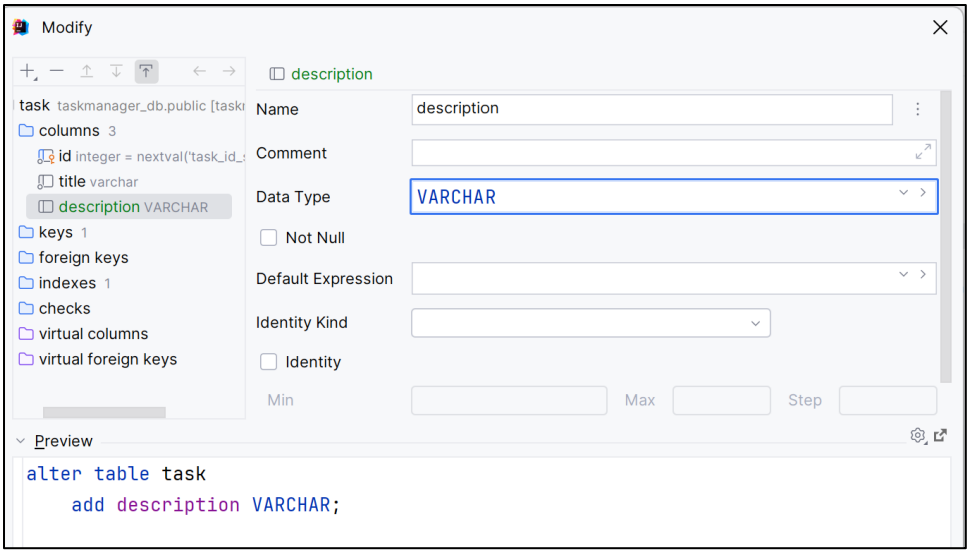
1. **id** (integer, primary key, serial (auto-generated):



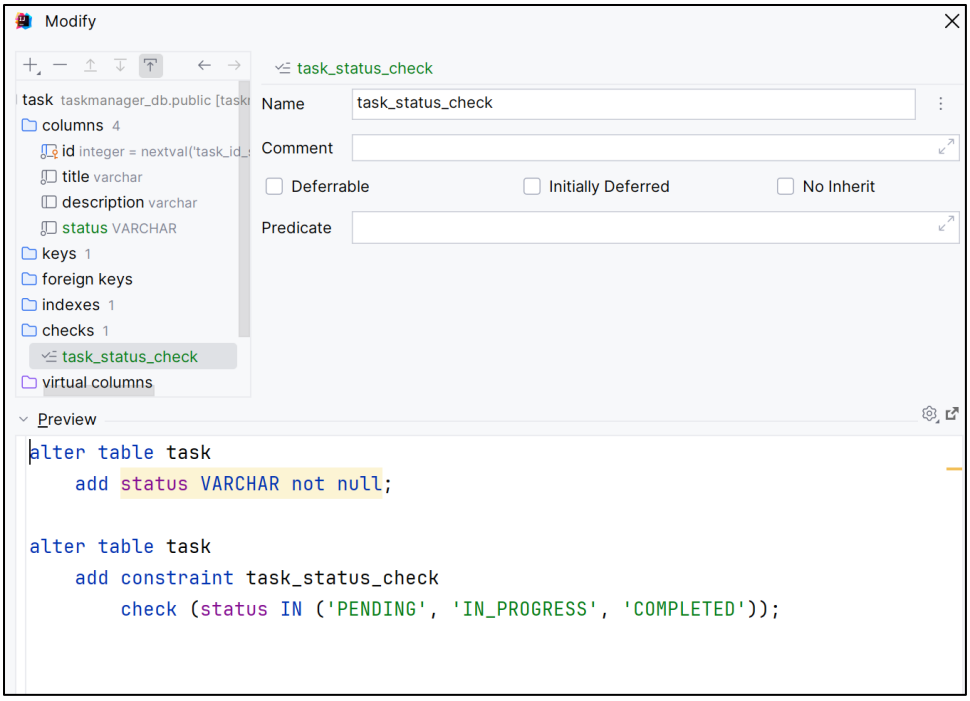
2. **title** (varchar, not null):
    - Description: a string representing the task's title
    - Example: "Complete project proposal", "Review design document"



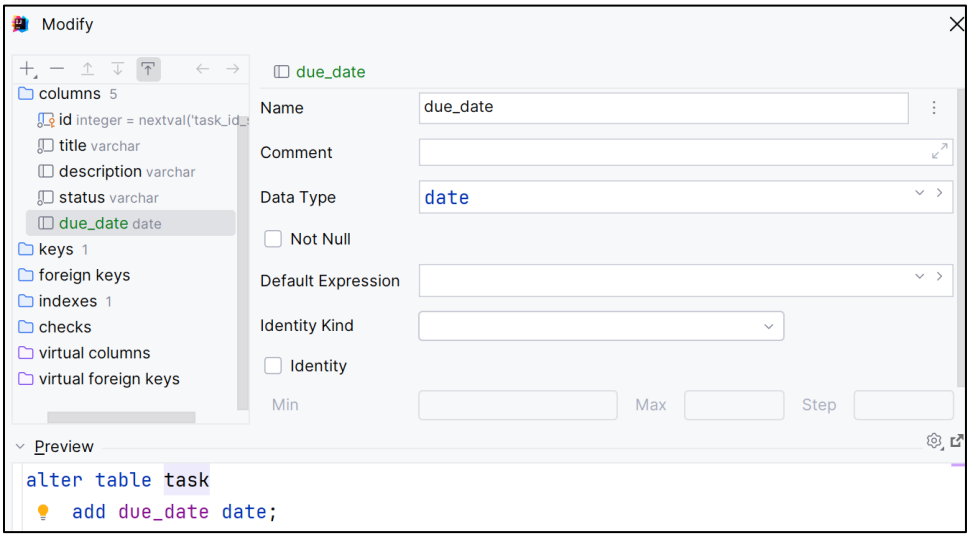
3. **description** (varchar, nullable):
    - Description: a string representing the description of the task
    - Example: "Prepare slides for the meeting" or NULL



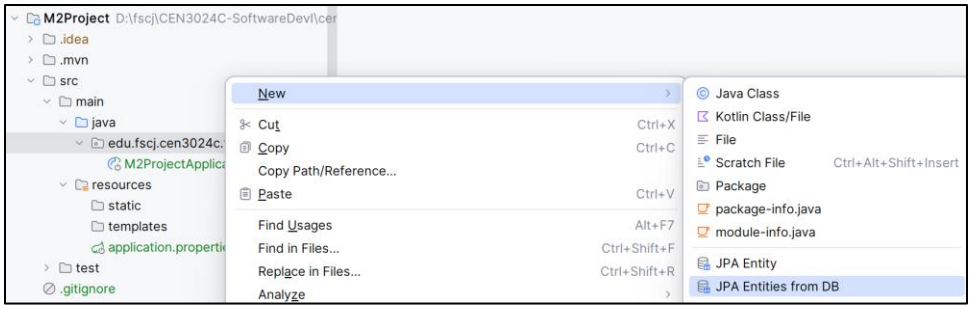
4. **status** (varchar w/Check, not null):
    - Description: the current status of the task, with three possible values: PENDING, IN_PROGRESS, or COMPLETED.

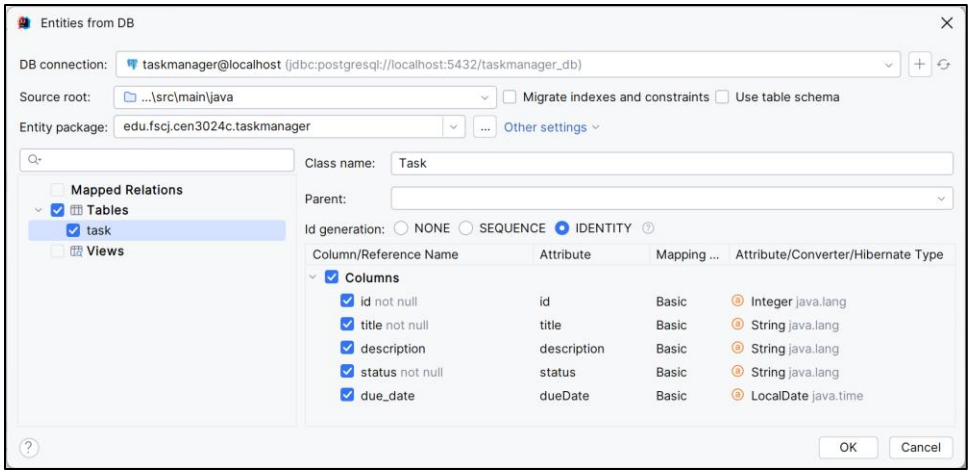


5. **due_date** (date, Nullable):
    - PostgreSQL Type: date
    - Description: the due date for the task in the format YYYY-MM-DD

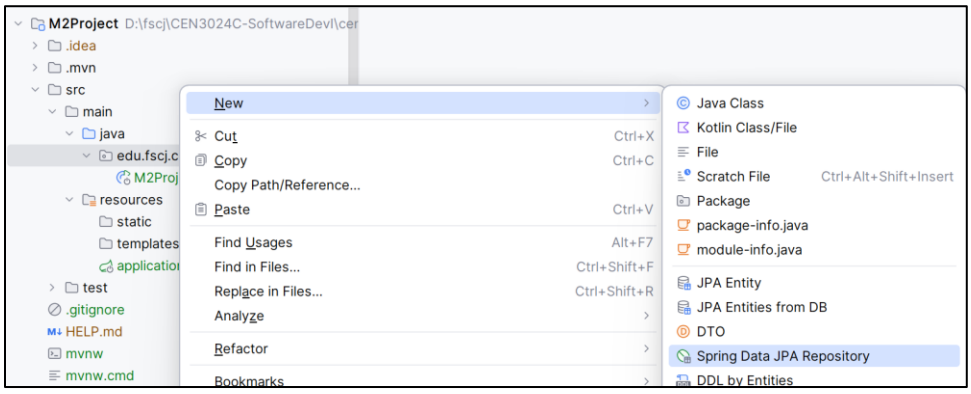


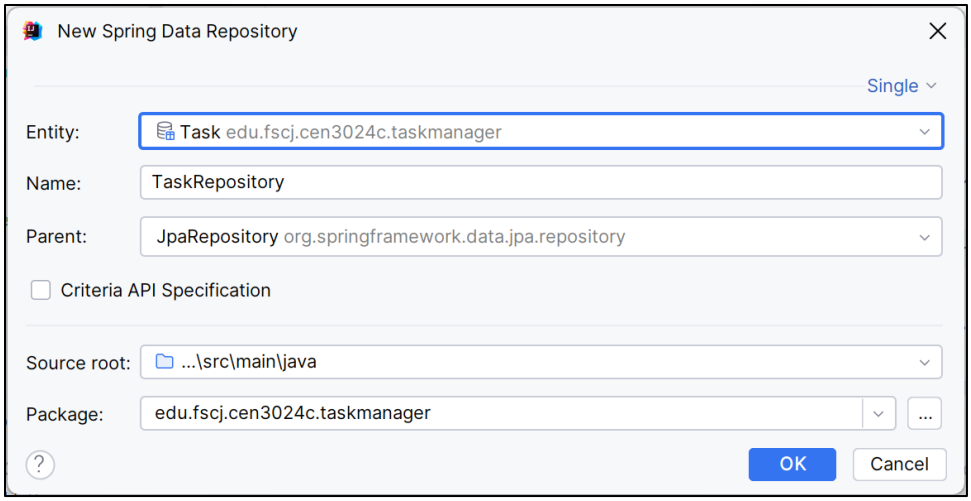
- Create a JPA Entity from the DB (this class will represent the database table in the application):





- Implement a TaskRepository interface that extends JpaRepository (an interface in Spring Data that provides CRUD operations for JPA entities without boilerplate code):





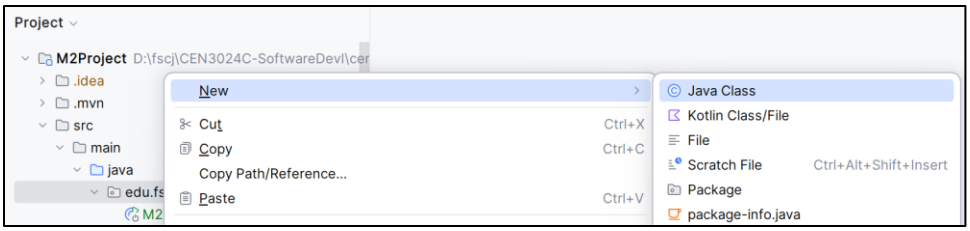
- Create a TaskService class (provides business logic, including creating, retrieving, updating, and deleting tasks):

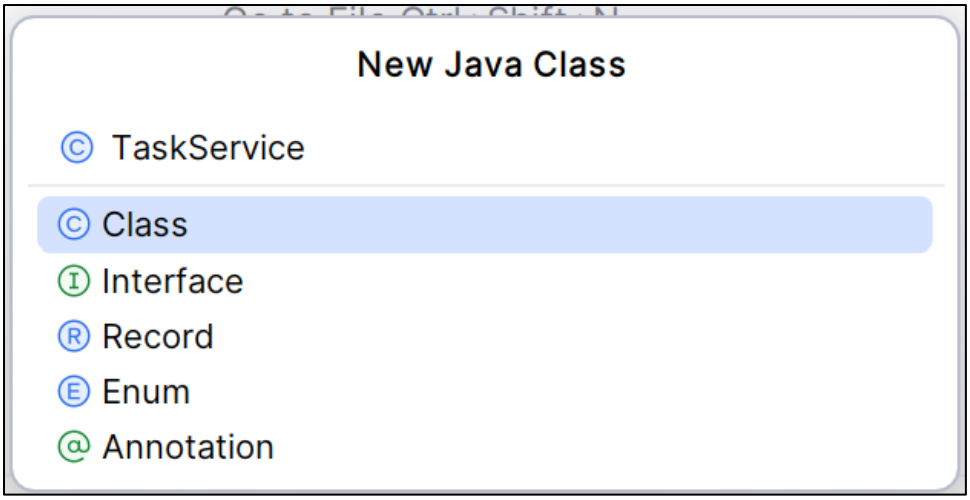
```
package edu.fscj.cen3024c.taskmanager;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class TaskService {

    @Autowired
    private TaskRepository taskRepository;

    public List<Task> findAll() {
        return taskRepository.findAll();
    }

    public Task findById(Integer id) {
        return taskRepository.findById(id)
                .orElseThrow(() -> new TaskNotFoundException(id));
    }

    public Task save(Task task) {
        return taskRepository.save(task);
    }

    public void deleteById(Integer id) {
```

```
        taskRepository.deleteById(id);
    }
}
```

- Add a new TaskNotFoundException class:

```
package edu.fscj.cen3024c.taskmanager;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class TaskNotFoundException extends RuntimeException {

    public TaskNotFoundException(Integer id) {
        super("Task not found with id " + id);
    }
}
```





- Implement a TaskController to expose REST endpoints (handles incoming HTTP requests, processes using the service class, and returns responses to the client):
    - GET /tasks: Retrieve all tasks

- GET /tasks/{id}: Retrieve a task by its ID
- POST /tasks: Create a new task
- PUT /tasks/{id}: Update an existing task
- DELETE /tasks/{id}: Delete a task by its ID

```
package edu.fscj.cen3024c.taskmanager;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/tasks")
public class TaskController {

    @Autowired
    private TaskService taskService;

    @GetMapping
    public List<Task> getAllTasks() {
        return taskService.findAll();
    }

    @GetMapping("/{id}")
    public Task getTaskById(@PathVariable Integer id) {
        return taskService.findById(id);
    }

    @PostMapping
    public Task createTask(@RequestBody Task task) {
        return taskService.save(task);
    }

    @PutMapping("/{id}")
    public Task updateTask(@PathVariable Integer id, @RequestBody Task
task) {
        Task existingTask = taskService.findById(id);
        existingTask.setTitle(task.getTitle());
        existingTask.setDescription(task.getDescription());
```

```
        existingTask.setStatus(task.getStatus());
        existingTask.setDueDate(task.getDueDate());
        return taskService.save(existingTask);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteTask(@PathVariable Integer id) {
        taskService.deleteById(id);
        return new ResponseEntity<>(HttpStatus.NO_CONTENT);
    }
}
```

- Start the application and use cURL in a Command Tool window to test the API endpoints. Take screen snips of results of executing the following and upload to your assignment repo.

    1. Create a task:

       ```
       curl -X POST http://localhost:8080/tasks ^
       -H "Content-Type: application/json" ^
       -d "{\"title\": \"New Task\", \"description\": \"Description of
       the task\", \"status\": \"pending\", \"dueDate\": \"2024-09-
       15\"}"
       ```

    2. Get all tasks:

       ```
       curl -X GET http://localhost:8080/tasks
       ```

    3. Get one task: (replace {id} with the id from the previous GET):

       ```
       curl -X GET http://localhost:8080/tasks/{id}
       ```

    4. Delete the task (replace {id}):

       ```
       curl -X DELETE http://localhost:8080/tasks/{id}
       ```

    Note that this results in our custom exception being thrown; the JSON includes a timestamp, status, and stack trace.

- Submit your project to your assignment repo; be sure include the screen snips with the CURL command results.